Capítulo 10 – Estrutura de Dados

PPGEC2316 – Algoritmos de Engenharia

Raphael Ramos da Silva

Universidade Federal do Rio Grande do Norte

20 de outubro de 2026

Referência: T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4ª ed., MIT Press, 2022 – Parte III.

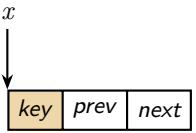
Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

- **Conjuntos dinâmicos**: conjuntos manipulados por algoritmos podem crescer, encolher ou mudar ao longo do tempo.
- **Dicionário:** um conjunto dinâmico que suporta inserir, remover e testar pertinência.
- Cada elemento é um objeto, acessado por meio de um ponteiro, contendo:
  - uma chave (*key*) identificadora;
  - atributos manipulados pelas operações (p. ex. ponteiros *prev*, *next*, etc).



objeto apontado por x
atributos: $x.key$, $x.prev$, $x.next$

Operações em conjuntos dinâmicos

- **Consultas** (*queries*)
  - $\textsc{Search}(S, k)$: retorna um ponteiro x para um elemento com $x.key = k$, ou NIL.
  - $\textsc{Minimum}(S)$, $\textsc{Maximum}(S)$: elemento de menor (maior) chave.
  - $\textsc{Successor}(S, x)$: próximo elemento maior que x, ou NIL se x é o máximo.
  - $\textsc{Predecessor}(S, x)$: próximo elemento menor que x, ou NIL se x é o mínimo.
- **Operações modificadoras**
  - $\textsc{Insert}(S, x)$: adiciona ao conjunto o elemento apontado por x.
  - $\textsc{Delete}(S, x)$: dado um *ponteiro* x (não uma chave), remove x de S.

Roteiro

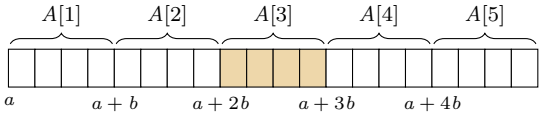
1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Arrays

- Um array é armazenado como uma sequência contígua de bytes na memória.
- Se o primeiro índice é s, o array começa no endereço a e cada elemento ocupa b bytes, o i-ésimo elemento ocupa os bytes

$$a + b(i - s) \quad \text{até} \quad a + b(i - s + 1) - 1.$$

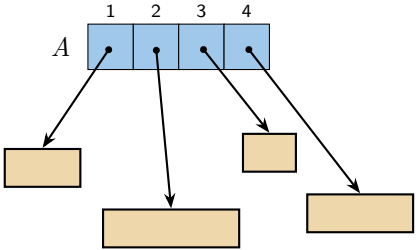
- $s = 1$: bytes $a + b(i-1)$ a $a + bi - 1$; $s = 0$: bytes $a + bi$ a $a + b(i+1) - 1$.
- No modelo RAM, acessar qualquer elemento leva tempo constante, independentemente do índice.



$s = 1$, $b = 4$:
$A[3]$ ocupa os bytes
$a + 2b$ a $a + 3b - 1$

Arrays de elementos de tamanhos diferentes

- A maioria das linguagens exige que todos os elementos de um array tenham o mesmo tamanho b.
- Se os objetos têm tamanhos variáveis, o array guarda ponteiros para os objetos (ponteiros têm tamanho fixo).
- As fórmulas anteriores dão o endereço do *ponteiro*; depois basta segui-lo até o objeto.



objetos de tamanhos variados

Matrizes

- Uma matriz $m \times n$ é normalmente representada por um ou mais arrays unidimensionais.
- Row-major: armazenada linha a linha. Column-major: coluna a coluna.

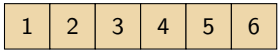
$$M = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \qquad \text{row-major: } 1\,2\,3\,4\,5\,6 \qquad \text{column-major: } 1\,4\,2\,5\,3\,6$$

- Com índices começando em s, o elemento $M[i, j]$ está na posição:

	row-major	column-major
geral	$s + n(i - s) + (j - s)$	$s + m(j - s) + (i - s)$
$s = 1$	$n(i - 1) + j$	$i + m(j - 1)$
$s = 0$	$ni + j$	$i + mj$

- Exemplo ($s = 1$): $M[2, 1]$ está na posição $3(2 - 1) + 1 = 4$ (row-major) e $2 + 2(1 - 1) = 2$ (column-major).

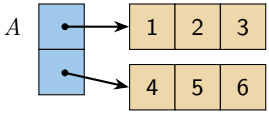
Matrizes: quatro formas de armazenar M



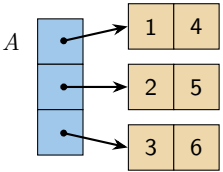
(a) row-major, array único



(b) column-major, array único



(c) row-major, um array por linha
$A[i][j] = M[i, j]$



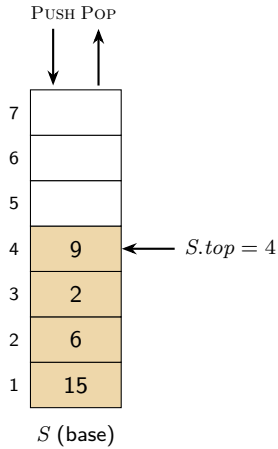
(d) column-major, um array por coluna
$A[j][i] = M[i, j]$

Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Stacks (pilhas)

- Conjunto dinâmico em que DELETE remove o elemento inserido mais recentemente: política LIFO (*last-in, first-out*).
- INSERT $\rightarrow$ PUSH; DELETE $\rightarrow$ POP (sem argumento de elemento).
- Implementação com array $S[1:n]$ e atributos:
  - $S.top$: índice do elemento mais recente;
  - $S.size$: tamanho n do array.
- A pilha é $S[1:S.top]$; $S[1]$ é a base e $S[S.top]$ o topo.
- $S.top = 0$: pilha vazia. Pop em pilha vazia: underflow; $S.top$ excedendo $S.size$: overflow.



Stacks: STACK-EMPTY, PUSH e POP

STACK-EMPTY(S)

```
1 if S.top == 0
2     return TRUE
3 else return FALSE
```

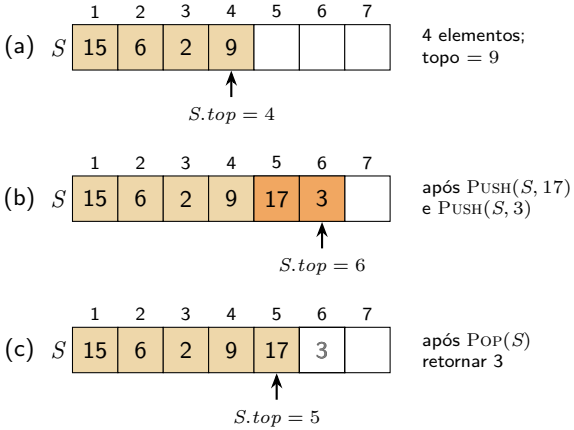
PUSH(S, x)

```
1 if S.top == S.size
2     error "overflow"
3 else S.top = S.top + 1
4     S[S.top] = x
```

POP(S)

```
1 if STACK-EMPTY(S)
2     error "underflow"
3 else S.top = S.top - 1
4     return S[S.top + 1]
```

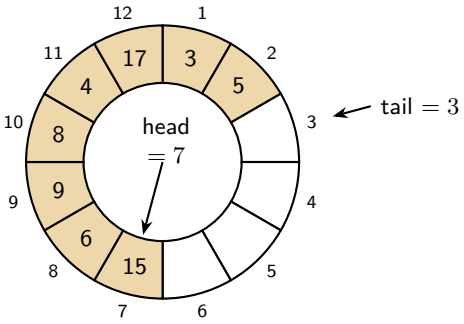
Cada operação executa em $O(1)$.



Apenas as posições em bege pertencem à pilha: o 3 continua no array em (c), mas não está mais na pilha.

Queues (filas)

- DELETE remove o elemento que está há mais tempo no conjunto: política FIFO (*first-in, first-out*).
- INSERT $\rightarrow$ ENQUEUE; DELETE $\rightarrow$ DEQUEUE.
- Array $Q[1:n]$, com $Q.size = n$, armazena até $n-1$ elementos:
  - $Q.head$: índice da cabeça;
  - $Q.tail$: próxima posição livre.
- Elementos em $Q.head, Q.head+1, \ldots, Q.tail-1$, com *wrap around*: a posição 1 segue a posição n.
- Vazia: $Q.head = Q.tail$ (inicialmente ambos $= 1$).
- Cheia: $Q.head = Q.tail + 1$, ou $Q.head = 1$ e $Q.tail = Q.size$.



Visão circular da fila $Q[1:12]$.

Queues: ENQUEUE e DEQUEUE

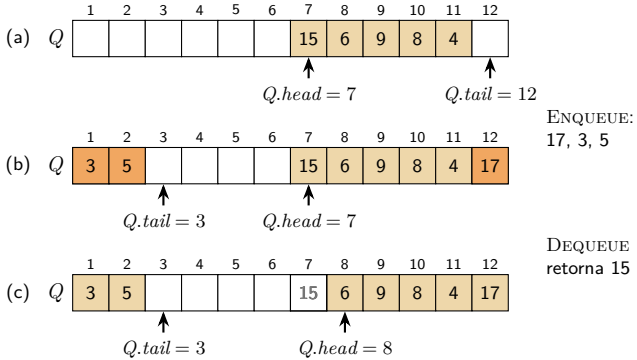
ENQUEUE(Q, x)

1. $Q[Q.tail] = x$
2. **if** $Q.tail == Q.size$
3. $\quad Q.tail = 1$
4. **else** $Q.tail = Q.tail + 1$

DEQUEUE(Q)

1. $x = Q[Q.head]$
2. **if** $Q.head == Q.size$
3. $\quad Q.head = 1$
4. **else** $Q.head = Q.head + 1$
5. **return** x

Cada operação: $O(1)$.

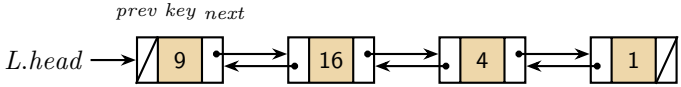


Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Linked lists (listas ligadas)

- Objetos organizados em ordem linear determinada por ponteiros em cada objeto (e não por índices, como no array).
- Lista duplamente ligada L: cada elemento tem key, $next$ e $prev$.
  - $x.prev = \text{NIL}$: x é a cabeça (*head*); $x.next = \text{NIL}$: x é a cauda (*tail*).
  - $L.head$ aponta para o primeiro elemento; $L.head = \text{NIL}$: lista vazia.
- Variações: simplesmente ou duplamente ligada; ordenada ou não; circular ou não.



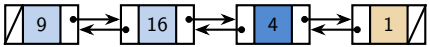
Busca: List-Search

List-Search(L, k)

```
1  x = L.head
2  while x ≠ NIL and x.key ≠ k
3      x = x.next
4  return x
```

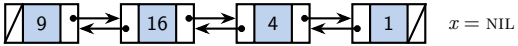
- Busca linear pelo primeiro elemento com chave k.
- Pior caso: $\Theta(n)$.

List-Search$(L, 4)$:



retorna x

List-Search$(L, 7)$:

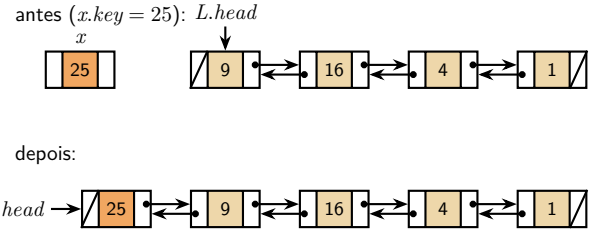


Inserção na cabeça: List-Prepend

List-Prepend(L, x)

1 $x.next = L.head$
2 $x.prev = \text{NIL}$
3 **if** $L.head \neq \text{NIL}$
4 $\quad L.head.prev = x$
5 $L.head = x$

Adiciona x (com $x.key$ já definido) na frente da lista em $O(1)$.



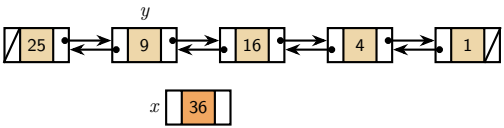
Inserção após um elemento: LIST-INSERT

LIST-INSERT(x, y)

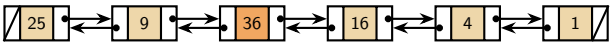
```
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```

"Emenda" x logo após y em $O(1)$. Não referencia a lista L.

antes ($x.key = 36$, y aponta para 9):



depois:



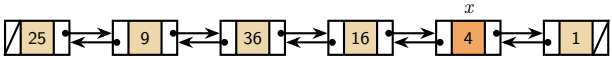
Remoção: List-Delete

List-Delete(L, x)

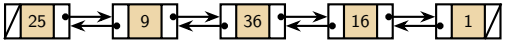
1 **if** $x.prev \neq$ NIL
2 $\quad x.prev.next = x.next$
3 **else** $L.head = x.next$
4 **if** $x.next \neq$ NIL
5 $\quad x.next.prev = x.prev$

- Recebe um *ponteiro* para x: $O(1)$.
- Remover por chave exige List-Search antes: $\Theta(n)$.

antes (x aponta para 4):



depois:



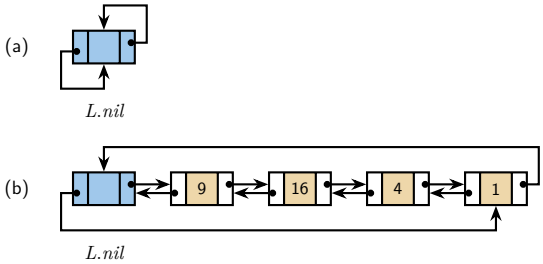


removido

Comparação: inserir/remover o primeiro elemento de um array custa $\Theta(n)$ (deslocamentos), contra $O(1)$ na lista duplamente ligada; já acessar o k-ésimo elemento custa $O(1)$ no array e $\Theta(k)$ na lista.

Sentinelas

- Uma sentinela (L) é um objeto fictício que remove a necessidade das condicionais do LIST-INSERT e LIST-DELETE
- $L.nil$ representa NIL, mas tem todos os atributos dos demais objetos.
- A lista se torna circular, duplamente ligada, com sentinela:
  - $L.nil.next$ aponta para a cabeça;
  - $L.nil.prev$ aponta para a cauda;
  - $L.head$ deixa de ser necessário.
- Lista vazia: só a sentinela, apontando para si mesma.



(a) lista vazia; (b) a lista $\langle 9, 16, 4, 1 \rangle$ com sentinela (azul).

Remoção e inserção com sentinela

LIST-DELETE$'(x)$

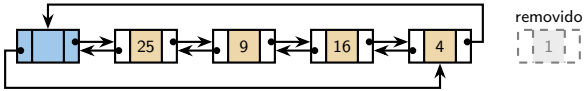
1. $x.prev.next = x.next$
2. $x.next.prev = x.prev$

LIST-INSERT$'(x, y)$

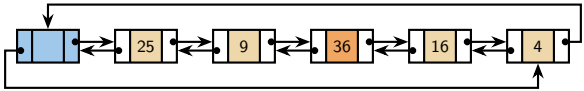
1. $x.next = y.next$
2. $x.prev = y$
3. $y.next.prev = x$
4. $y.next = x$

Inserir na cabeça: $y = L.nil$; na cauda: $y = L.nil.prev$. Nunca remova a sentinela!

LIST-DELETE$'(x)$, x aponta para 1:



LIST-INSERT$'(x, y)$, $x.key = 36$, y aponta para 9:



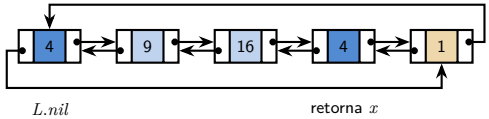
Busca com sentinela: LIST-SEARCH′

LIST-SEARCH′(L, k)

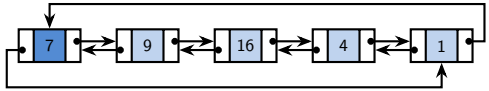
```
1  L.nil.key = k                  // store the key in the sentinel
2  x = L.nil.next                 // start at the head of the list
3  while x.key ≠ k
4      x = x.next
5  if x == L.nil                  // found k in the sentinel
6      return NIL                 // k was not really in the list
7  else return x                  // found k in element x
```

- A chave é garantidamente encontrada (na lista ou na sentinela): uma comparação a menos por iteração.
- Mesmo tempo assintótico, constante menor.

LIST-SEARCH′$(L, 4)$:



LIST-SEARCH′$(L, 7)$:



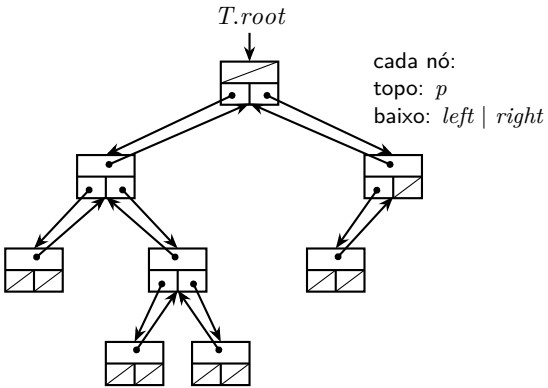
$x = L.nil \Rightarrow$ NIL

Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Árvores binárias

- Cada nó é um objeto com key e ponteiros:
  - p: pai;
  - $left$: filho esquerdo;
  - $right$: filho direito.
- $x.p = \text{NIL}$: x é a raiz.
- $x.left = x.right = \text{NIL}$: x é uma folha.
- $T.root$ aponta para a raiz;
  $T.root = \text{NIL}$: árvore vazia.

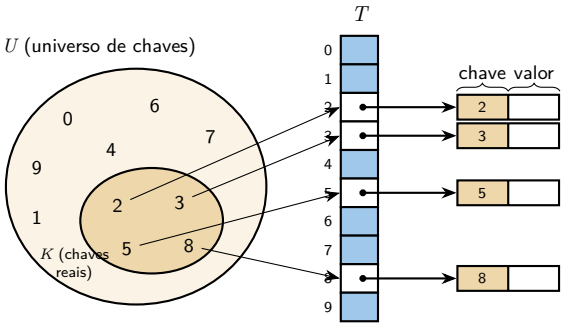


Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Tabelas de endereçamento direto

- Considere um conjunto universo $U = \{0, 1, \ldots, m-1\}$ de chaves.
- Tabela de endereçamento direto $T[0 : m-1]$ mapeia cada posição corresponde a uma chave de U.
- Desvantagem: as chaves $k \in U$ precisam existir na memória, mesmo que elas nunca sejam utilizadas na sua aplicação



Endereçamento direto: operações

DIRECT-ADDRESS-SEARCH(T, k)

1 **return** $T[k]$

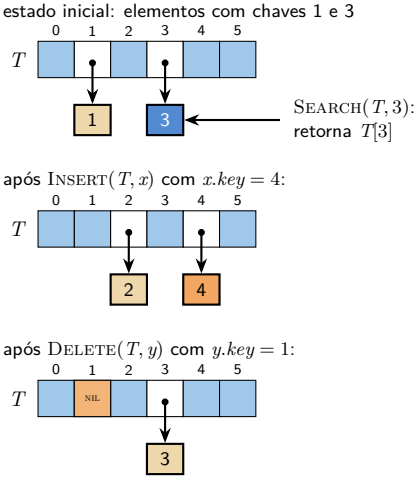
DIRECT-ADDRESS-INSERT(T, x)

1 $T[x.key] = x$

DIRECT-ADDRESS-DELETE(T, x)

1 $T[x.key] = \text{NIL}$

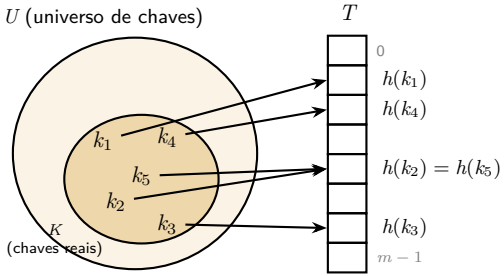
Cada operação: $O(1)$ no pior caso.



- Se U é grande, uma tabela de tamanho $|U|$ é impraticável; e o conjunto K de chaves armazenadas é muito menor que U.
- Uma função hash calcula o slot a partir da chave:

$$h : U \to \{0, 1, \ldots, m-1\}, \qquad m \ll |U|.$$

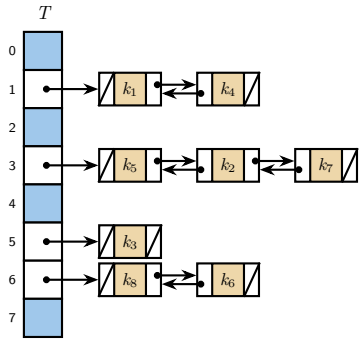
- O elemento de chave k vai para o slot $h(k)$ (*valor hash* de k).
- Armazenamento: $\Theta(|K|)$, busca em $O(1)$ em média.
- Colisão: duas chaves com o mesmo valor hash. Como $|U| > m$, colisões são inevitáveis.



para $h(k) = k \bmod m$, k_2 e k_5 colidem

Encadeamento (*chaining*)

- O encadeamento (*chaining*) é uma forma de lidar com colisões
- Cada slot $T[j]$ aponta para uma lista ligada com todos os elementos cujo valor hash é j (NIL se não houver).



$$h(k_1) = h(k_4) = 1$$
$$h(k_5) = h(k_2) = h(k_7) = 3$$

Operações com encadeamento

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND$(T[h(x.key)], x)$

CHAINED-HASH-SEARCH(T, k)

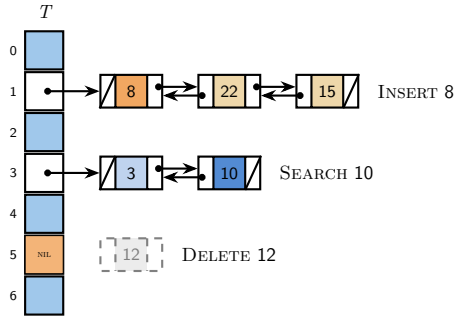
1 **return** LIST-SEARCH$(T[h(k)], k)$

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE$(T[h(x.key)], x)$

- Insert: $O(1)$ no pior caso (supõe x ausente).
- Search: proporcional ao tamanho da lista.
- Delete: $O(1)$ com listas duplamente ligadas.

$m = 7$, $h(k) = k \bmod 7$, após inserir 15, 22, 12, 10, 3:



Pior caso

Todas as n chaves no mesmo slot: busca $\Theta(n)$, em que n número de elementos armazenados

Caso médio

Seja $\alpha = n/m$ (fator de carga). Se a função hash espalha bem as chaves, cada lista tem, em média, α elementos. Buscar uma chave custa: 1 (calcular o hash e ir ao slot) $+ \alpha$ (percorrer a lista).

- busca *sem sucesso*: percorre a lista inteira $\Rightarrow \Theta(1 + \alpha)$;
- busca *com sucesso*: para no meio da lista, em média $\Rightarrow$ também $\Theta(1 + \alpha)$.

Se $n = O(m)$, então $\alpha = O(1)$: todas as operações de dicionário em $O(1)$ em média.

Método da divisão

$$h(k) = k \bmod m.$$

Ex.: $m = 12$, $k = 100 \Rightarrow h(k) = 4$

Método da multiplicação

$$h(k) = \lfloor m\,(kA \bmod 1) \rfloor, \qquad 0 < A < 1,$$

em que $kA \bmod 1 = kA - \lfloor kA \rfloor$ é a parte fracionária de kA.
Ex.: $m = 12$, $k = 100$, $A = (\sqrt{5} - 1)/2 \approx 0{,}618 \Rightarrow kA \approx 61{,}803$, $kA \bmod 1 \approx 0{,}803$ e
$h(k) = \lfloor 12 \cdot 0{,}803 \rfloor = \lfloor 9{,}64 \rfloor = 9$

Endereçamento aberto (*open addressing*)

- Trata-se de uma outra maneira de lidar com colisões.
- Diferentemente do **encadeamento**, não há necessidade de sublistas: todos os elementos ficam na própria tabela
- Cada *slot* contém um elemento ou NIL(vazio)
- Para inserir uma chave k, sondamos (*probe*) a tabela até achar um slot vazio. Esse processo é feito na sequência de sondagem da chave k, definida por

$$\langle h(k,0), h(k,1), \ldots, h(k,m-1) \rangle$$

onde $h(k,i)$ é a i-ésima tentativa (sondagem) para inserir a chave k.

Como implementar a sondagem?

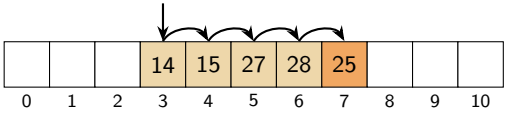
A sequência de sondagem $h(k, i)$ é construída a partir de funções hash auxiliares. Há duas formas de fazê-la: com duas funções ou apenas uma.

- **Double hashing**: duas funções h_1 e h_2, tal que $h(k, i) = (h_1(k) + i\,h_2(k)) \bmod m$
  - $h_1(k)$ dá a primeira sonda; $h_2(k)$ dá o passo entre sondas.
- **Linear probing**: uma função, tal que $h(k, i) = (h_1(k) + i) \bmod m$
  - Começa em $h_1(k)$ e sonda os slots consecutivos $h_1(k) + 1, h_1(k) + 2, \ldots$, até atingir o limite da tabela, onde volta-se para o primeiro índice (*wrap around*).
  - Sequências longas de slots ocupados tendem a crescer.

Sondagem: *linear probing* × *double hashing*

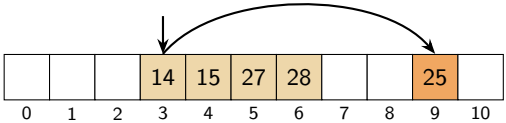
$m = 11$, $h_1(k) = k \bmod 11$, $h_2(k) = 1 + (k \bmod 10)$. HASH-INSERT$(T, 25)$: $h_1(25) = 3$, $h_2(25) = 6$.

Linear probing: $h(25, i) = (3 + i) \bmod 11$



sondas: 3, 4, 5, 6, 7
4 colisões ⇒ slot 7

Double hashing: $h(25, i) = (3 + 6i) \bmod 11$



sondas: 3, 9
1 colisão ⇒ slot 9

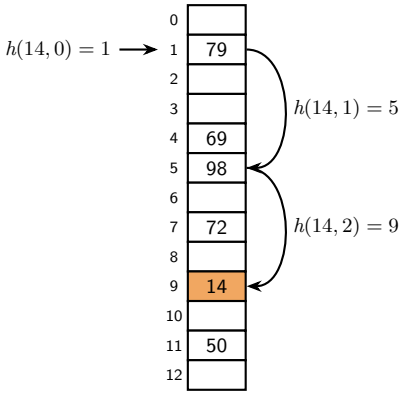
No *linear probing*, 25 percorre todo o bloco de slots ocupados 3, 4, 5, 6;
no *double hashing*, o passo 6 salta sobre esse bloco.

Inserção: HASH-INSERT

HASH-INSERT(T, k)

```
1  i = 0
2  repeat
3      q = h(k, i)
4      if T[q] == NIL
5          T[q] = k
6          return q
7      else i = i + 1
8  until i == m
9  error "hash table overflow"
```

Retorna o slot onde k foi armazenada ou sinaliza tabela cheia.



Hash-Insert$(T, 14)$

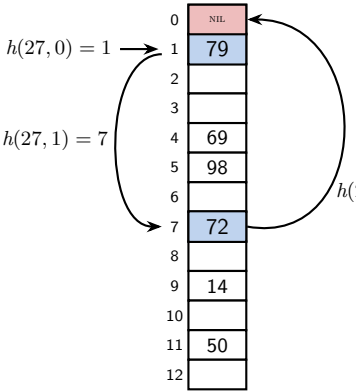
$m = 13$
$h_1(k) = k \bmod 13$
$h_2(k) = 1 + (k \bmod 11)$
$h(k, i) = (h_1(k) + i\,h_2(k)) \bmod 13$

Busca: HASH-SEARCH

HASH-SEARCH(T, k)

```
1  i = 0
2  repeat
3      q = h(k, i)
4      if T[q] == k
5          return q
6      i = i + 1
7  until T[q] == NIL  or i == m
8  return NIL
```

Percorre a mesma sequência da inserção: encontrar um slot vazio prova que k não está na tabela.



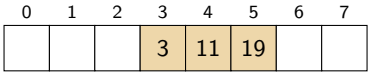
Hash-Search(T, 27)

$m = 13$
$h_1(k) = k \bmod 13$
$h_2(k) = 1 + (k \bmod 11)$
$h(k,i) = (h_1(k) + i\,h_2(k)) \bmod 13$

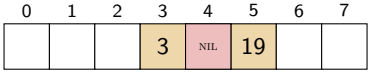
Remoção em endereçamento aberto

- Remover a chave do slot q marcando-o com NIL é um erro: chaves inseridas depois, cuja sondagem passou por q, se tornariam inacessíveis.
- Solução: marcar o slot com o valor especial DELETED.
  - HASH-INSERT trata DELETED como vazio;
  - HASH-SEARCH passa por cima de DELETED.
- Desvantagem: o tempo de busca deixa de depender só de $\alpha \Rightarrow$ encadeamento é preferido quando há muitas remoções.

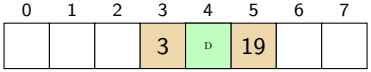
$$h(k, i) = (k + i) \bmod 8$$



chaves 3, 11, 19
colidem no slot 3



remover 11 com NIL:
busca por 19 para no slot 4!



remover 11 com DELETED:
a busca continua e acha 19

Pior caso

A sequência de sondagem visita todas as m posições: busca e inserção em $\Theta(m)$, em que m é o tamanho da tabela

Caso médio

Seja $\alpha = n/m < 1$ (fator de carga). Com hashing uniforme, cada sonda encontra uma posição ocupada com probabilidade $\leq \alpha$. A i-ésima sonda só ocorre se as $i-1$ anteriores acharam posições ocupadas (probabilidade $\leq \alpha^{i-1}$).

- busca *sem sucesso*: número esperado de sondas $\leq 1 + \alpha + \alpha^2 + \cdots = \frac{1}{1-\alpha}$;
- INSERT: busca sem sucesso seguida da escrita na posição vazia $\Rightarrow$ também $\leq \frac{1}{1-\alpha}$.

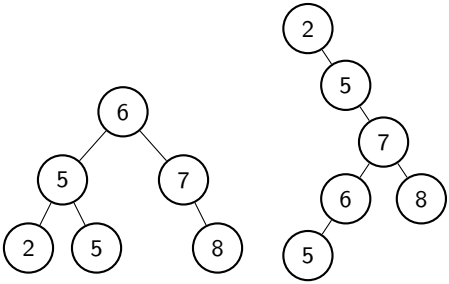
Se α é constante, então busca e inserção em $O(1)$ em média (ex.: $\alpha = 0{,}5 \Rightarrow \leq 2$ sondas; $\alpha = 0{,}9 \Rightarrow \leq 10$ sondas).

Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Binary search trees (árvores binárias de busca)

- Suportam SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT e DELETE: servem como dicionário e como fila de prioridade.
- Cada nó: key, dados satélites, $left$, $right$ e p.
- Propriedade de BST: se y está na subárvore esquerda de x, $y.key \leq x.key$; se está na direita, $y.key \geq x.key$.
- Operações em tempo $O(h)$, $h =$ altura: $\Theta(\lg n)$ para árvore completa, mas $\Theta(n)$ se for uma cadeia linear.



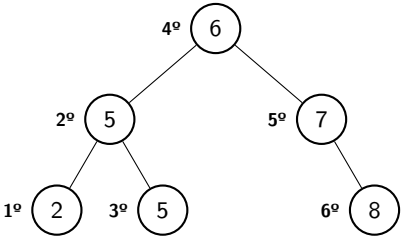
(a) altura 2 (b) altura 4

Duas BSTs com as mesmas chaves.

Percurso em ordem: INORDER-TREE-WALK

```
INORDER-TREE-WALK(x)
1  if x ≠ NIL
2      INORDER-TREE-WALK(x.left)
3      print x.key
4      INORDER-TREE-WALK(x.right)
```

- Imprime a raiz *entre* as subárvores esquerda e direita: chaves em ordem crescente.
- *Preorder*: raiz antes; *postorder*: raiz depois.
- Chamada inicial: INORDER-TREE-WALK($T.root$); tempo $\Theta(n)$.



ordem de impressão: 2, 5, 5, 6, 7, 8

Busca: Tree-Search

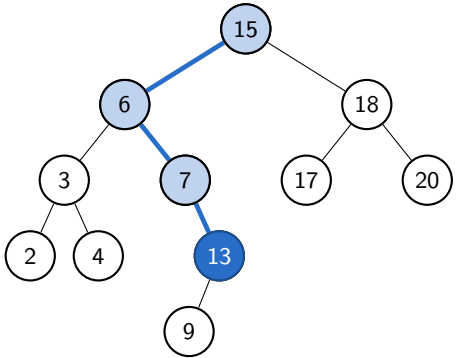
Tree-Search(x, k)

```
1  if x == NIL  or k == x.key
2      return x
3  if k < x.key
4      return Tree-Search(x.left, k)
5  else return Tree-Search(x.right, k)
```

Iterative-Tree-Search(x, k)

```
1  while x ≠ NIL  and k ≠ x.key
2      if k < x.key
3          x = x.left
4      else x = x.right
5  return x
```

Caminho simples descendente: $O(h)$.



Tree-Search$(T.root, 13)$: $15 \to 6 \to 7 \to 13$

Mínimo e máximo: TREE-MINIMUM e TREE-MAXIMUM

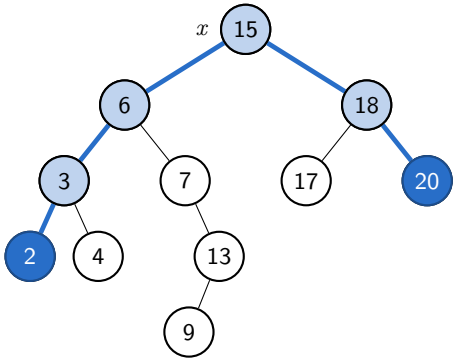
TREE-MINIMUM(x)

```
1  while x.left ≠ NIL
2      x = x.left
3  return x
```

TREE-MAXIMUM(x)

```
1  while x.right ≠ NIL
2      x = x.right
3  return x
```

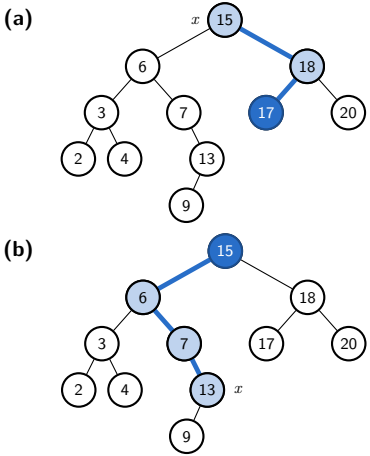
Seguir ponteiros *left* (ou *right*) a partir da raiz; $O(h)$. Supõe $x \neq$ NIL.



mínimo $= 2$, máximo $= 20$

TREE-SUCCESSOR(x)

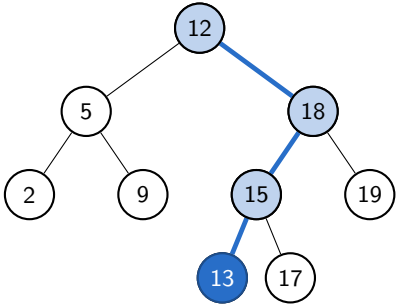
```
1  if x.right ≠ NIL
2      return TREE-MINIMUM(x.right)
3  else
4      y = x.p
5      while y ≠ NIL  and x == y.right
6          x = y
7          y = y.p
8      return y
```



- **(a)** O sucessor de 15 é 17 (o mínimo da subárvore direita de 15)
- **(b)** O sucessor de 13 é 15: como 13 não tem filho direito, subimos a árvore até encontrar um nó que seja filho esquerdo de seu pai (6); o pai dele (15) é o sucessor.

TREE-INSERT(T, z)

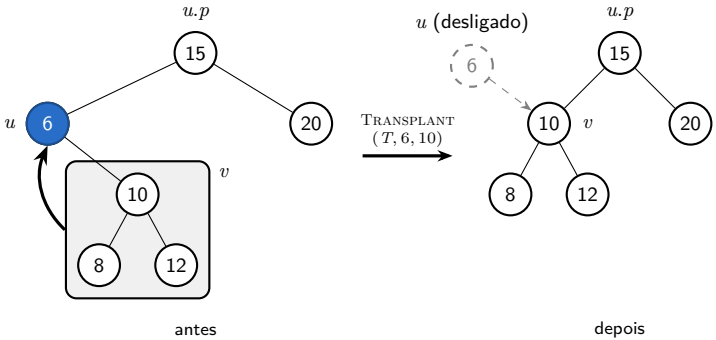
```
1  x = T.root                  // node being compared with z
2  y = NIL                     // y will be parent of z
3  while x ≠ NIL               // descend until reaching a leaf
4      y = x
5      if z.key < x.key
6          x = x.left
7      else x = x.right
8  z.p = y                     // found the location
9  if y == NIL
10     T.root = z              // tree T was empty
11 elseif z.key < y.key
12     y.left = z
13 else y.right = z
```



Inserção da chave 13: caminho $12 \to 18 \to 15$ (azul claro); novo nó em azul escuro. Tempo $O(h)$.

Sub-rotina TRANSPLANT

```
TRANSPLANT(T, u, v)
1  if u.p == NIL
2      T.root = v
3  elseif u == u.p.left
4      u.p.left = v
5  else u.p.right = v
6  if v ≠ NIL
7      v.p = u.p
```



Dica

Intuitivamente, transplante é "mover" a subárvore v para u.

Casos de remoção em uma BST

- Existem quatro casos de remoção de um nó z em uma BST, que serão comentados nos slides a seguir:
  - **Caso 1:** z sem filho esquerdo: substitui z por seu filho direito (talvez NIL);
  - **Caso 2:** z só com filho esquerdo: substitui z por esse filho;
  - **Caso 3:** dois filhos e o sucessor y é $z.right$: substitui z por y;
  - **Caso 4:** dois filhos e $y \neq z.right$: primeiro substitui y por seu filho direito, depois substitui z por y.

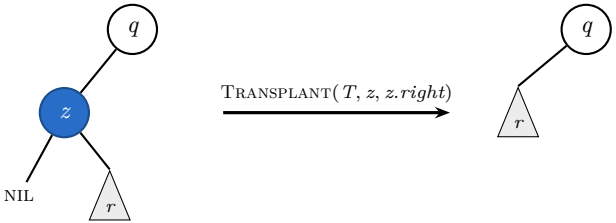
Remoção: TREE-DELETE

```
TREE-DELETE(T, z)
1  if z.left == NIL
2      TRANSPLANT(T, z, z.right)                  // replace z by its right child
3  elseif z.right == NIL
4      TRANSPLANT(T, z, z.left)                   // replace z by its left child
5  else y = TREE-MINIMUM(z.right)                 // y is z's successor
6      if y ≠ z.right                             // is y farther down the tree?
7          TRANSPLANT(T, y, y.right)              // replace y by its right child
8          y.right = z.right                      // z's right child becomes
9          y.right.p = y                          //    y's right child
10     TRANSPLANT(T, z, y)                        // replace z by its successor y
11     y.left = z.left                            // and give z's left child to y,
12     y.left.p = y                               //    which had no left child
```

Tudo é $O(1)$ exceto TREE-MINIMUM: $O(h)$.

Caso 1 da remoção

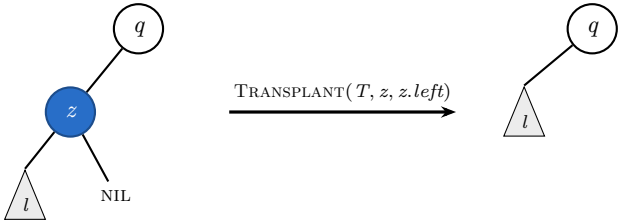
- z não tem filho esquerdo.
- Substituímos z por seu filho direito r, que pode ser NIL: $\textsc{Transplant}(T, z, z.right)$.



- Corresponde às linhas 1–2 de TREE-DELETE. Se z é uma folha, também cai neste caso (z é substituído por NIL).

Caso 2 da remoção

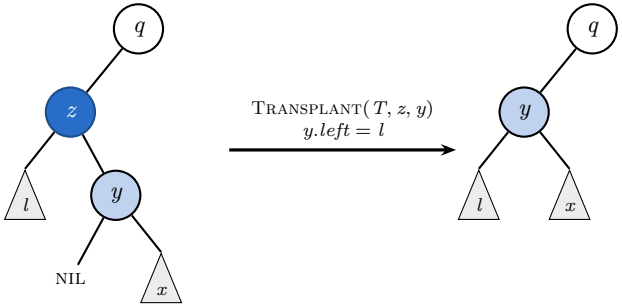
- z tem apenas o filho esquerdo.
- Substituímos z por seu filho esquerdo l: $\textsc{Transplant}(T, z, z.left)$.



- Corresponde às linhas 3–4 de $\textsc{Tree-Delete}$.

Caso 3 da remoção

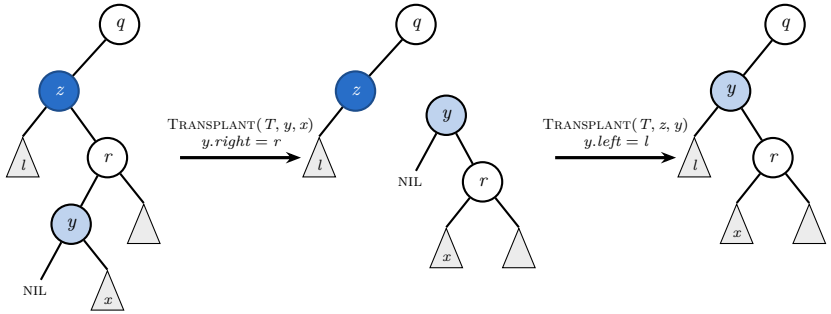
- z tem dois filhos e seu sucessor y é o próprio $z.right$ (y não tem filho esquerdo).
- Substituímos z por y, que herda o filho esquerdo l de z: $\textsc{Transplant}(T, z, y)$ e $y.left = l$.



- Corresponde às linhas 5 e 10–12 de $\textsc{Tree-Delete}$ (o teste da linha 6 falha).

Caso 4 da remoção

- z tem dois filhos e seu sucessor y está mais abaixo na subárvore direita ($y \neq z.right$).
- Primeiro substituímos y por seu filho direito x e fazemos $r = z.right$ virar filho direito de y.
- Depois substituímos z por y, que herda o filho esquerdo l de z.



- Corresponde às linhas 5–12 de TREE-DELETE.

Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Red-black trees (árvores rubro-negras)

- BSTs balanceadas: com n nós, a árvore tem altura máxima $\lg(n)$.
- Um bit extra por nó: $color \in \{\text{RED}, \text{BLACK}\}$. Atributos: $color$, key, $left$, $right$, p.

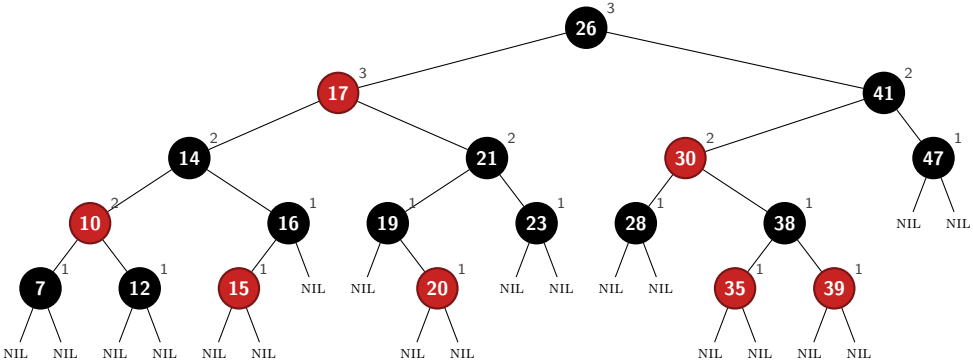
Propriedades rubro-negras

1. Todo nó é vermelho ou preto.
2. A raiz e as folhas (NIL) são pretas.
3. Se um nó é vermelho, ambos os filhos são pretos.
4. Para cada nó, todos os caminhos simples dele até folhas descendentes contêm o mesmo número de nós pretos.

Altura negra ($bh(x)$)

Número de nós pretos em qualquer caminho de x (exclusivo) até uma folha.

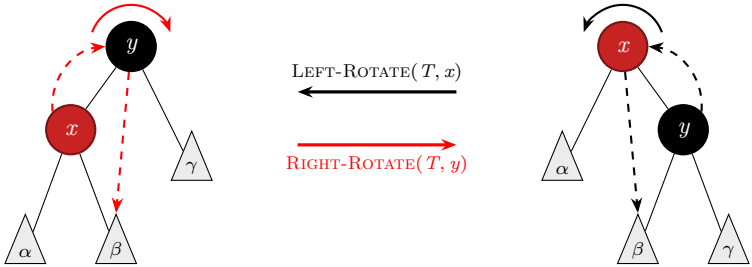
Exemplo de uma árvore rubro-negra



- A altura negra está apresentada a direita de cada nó.
- A propriedade 4 é satisfeita: por exemplo, todos os caminhos possíveis de 21 até as folhas NIL contêm 1 nó preto (19 ou 23) antes de chegar à folha.

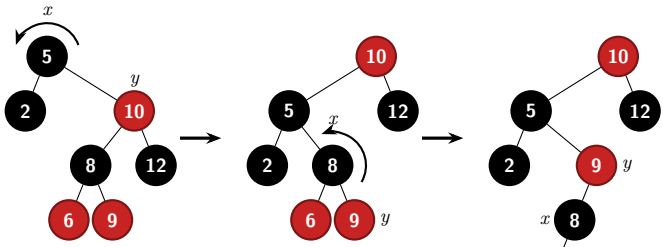
Rotações

- A rotação (para esquerda ou para direita) é um mudança local que altera apenas alguns ponteiros e é feita em $O(1)$.
- Após as rotações, as propriedades de BST se mantêm: $\alpha < x < \beta < y < \gamma$ antes e depois.



LEFT-ROTATE(T, x)

```
1  y = x.right
2  x.right = y.left
3  if y.left ≠ T.nil
4      y.left.p = x
5  y.p = x.p
6  if x.p == T.nil
7      T.root = y
8  elseif x == x.p.left
9      x.p.left = y
10 else x.p.right = y
11 y.left = x
12 x.p = y
```



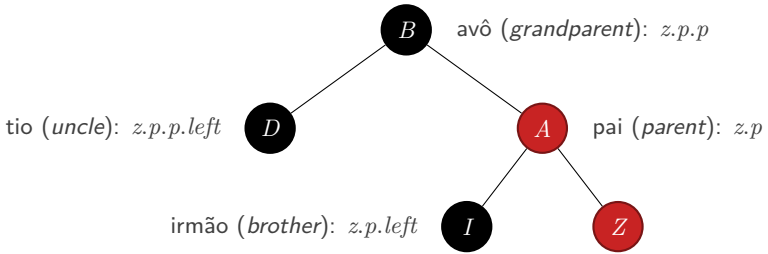
(a) inicial

(b) após LEFT-ROTATE$(T, 5)$

(c) após LEFT-ROTATE$(T, 8)$

Hierarquia de nós em uma BST

- Antes de comentar sobre inserção e remoção em árvores rubro-negras, falemos da hierarquia de nós em uma BST.
- Tomando como referência o nó Z: B é o avô (*grandparent*) de Z ($z.p.p$), A é o pai (*parent*) de Z ($z.p$) e D é o tio (*uncle*) de Z ($z.p.p.left$) e I é o irmão (*brother*) de Z ($z.p.left$).



- Existem quatro casos de inserção, que serão comentados nos slides a seguir:
  - **Caso 0:** z é a raiz;
  - **Caso 1:** o tio de z é vermelho;
  - **Caso 2:** o tio de z é preto (alinhamento em triângulo);
  - **Caso 3:** o tio de z é preto (alinhamento em linha).

Considerações

1. Um novo nó z é sempre inserido na cor vermelha.
2. RB-INSERT faz a inserção do nó respeitando a propriedade de BST, e RB-INSERT-FIXUP trata os casos acima.

RB-Insert e RB-Insert-Fixup

RB-Insert(T, z)

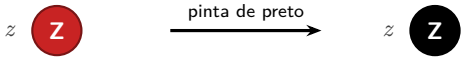
```
1  x = T.root
2  y = T.nil
3  while x ≠ T.nil
4      y = x
5      if z.key < x.key
6          x = x.left
7      else x = x.right
8  z.p = y
9  if y == T.nil
10     T.root = z
11 elseif z.key < y.key
12     y.left = z
13 else y.right = z
14 z.left = T.nil
15 z.right = T.nil
16 z.color = RED
17 RB-Insert-Fixup(T, z)
```

RB-Insert-Fixup(T, z)

```
1  while z.p.color == RED
2      if z.p == z.p.p.left
3          y = z.p.p.right                          // y é o tio de z
4          if y.color == RED
5              z.p.color = BLACK                    // caso 1
6              y.color = BLACK                      // caso 1
7              z.p.p.color = RED                    // caso 1
8              z = z.p.p                            // caso 1
9          else
10             if z == z.p.right                    // caso 2
11                 z = z.p                          // caso 2
12                 Left-Rotate(T, z)                // caso 2
13             z.p.color = BLACK                    // caso 3
14             z.p.p.color = RED                    // caso 3
15             Right-Rotate(T, z.p.p)               // caso 3
16     else            // igual às linhas 3–15, trocando left e right
17 T.root.color = BLACK                             // caso 0
```

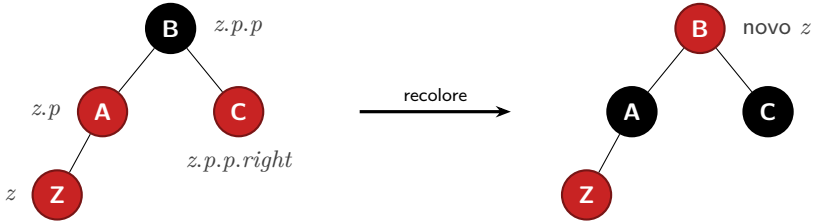
Caso 0 da inserção

- z é a raiz.
- Simplesmente pintamos z de preto.



Caso 1 da inserção

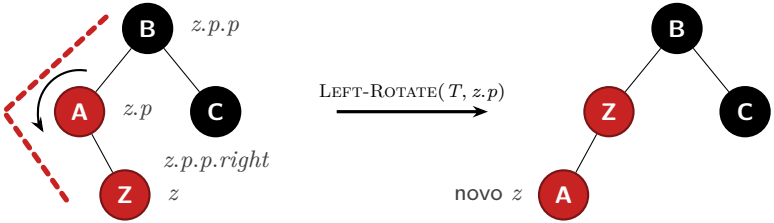
- $z.uncle$ é vermelho.
- Trocamos as cores do pai ($z.p$), do avô ($z.p.p$) e do tio ($z.p.p.right$).



- A ilustração considera $z.p = z.p.p.left$ (linhas 2–15 de RB-INSERT-FIXUP); logo, o tio é $z.p.p.right$.

Caso 2 da inserção

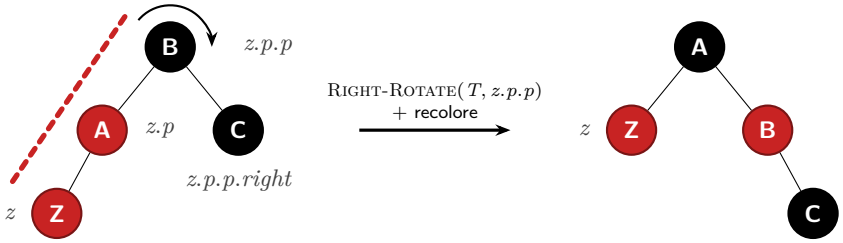
- $z.uncle$ é preto e z, $z.p$ e $z.p.p$ estão em alinhamento triângulo.
- Rotacionamos o pai ($z.p$) na direção oposta a z; aqui, LEFT-ROTATE($T, z.p$).



- A ilustração considera $z.p = z.p.p.left$ (linhas 2–15 de RB-INSERT-FIXUP); logo, o tio é $z.p.p.right$.

Caso 3 da inserção

- $z.uncle$ é preto e z, $z.p$ e $z.p.p$ estão em alinhamento linha.
- Rotacionamos o avô ($z.p.p$) na direção oposta a z; aqui, RIGHT-ROTATE($T, z.p.p$).
- Trocamos as cores do pai e do avô originais: A fica preto e B fica vermelho.



- A ilustração considera $z.p = z.p.p.left$ (linhas 2–15 de RB-INSERT-FIXUP); logo, o tio é $z.p.p.right$.

Remoção em árvores rubro-negras

- A remoção de um nó z contempla três casos, que serão comentados nos slides a seguir:
  - **Caso I:** o filho esquerdo de z é NIL;
  - **Caso II:** o filho direito de z é NIL;
  - **Caso III:** nenhum dos filhos de z é NIL.

Considerações

1. O método TRANSPLANT aqui é chamado de RB-TRANSPLANT (usa $T.nil$ no lugar de NIL).
2. RB-DELETE remove o nó z, e RB-DELETE-FIXUP corrige as violações de cores e das propriedades de árvore rubro-negra.

RB-Delete e RB-Delete-Fixup

RB-Delete(T, z)

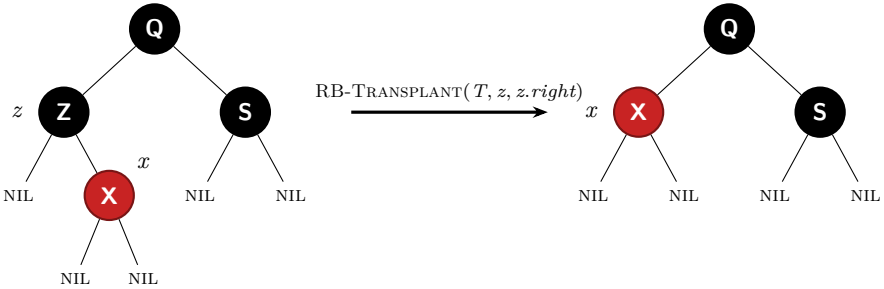
```
1  y = z
2  y-original-color = y.color
3  if z.left == T.nil                                  // caso I
4      x = z.right                                     // caso I
5      RB-Transplant(T, z, z.right)                    // caso I
6  elseif z.right == T.nil                             // caso II
7      x = z.left                                      // caso II
8      RB-Transplant(T, z, z.left)                     // caso II
9  else y = Tree-Minimum(z.right)                      // caso III
10     y-original-color = y.color                      // caso III
11     x = y.right                                     // caso III
12     if y ≠ z.right                                  // caso III
13         RB-Transplant(T, y, y.right)                // caso III
14         y.right = z.right                           // caso III
15         y.right.p = y                               // caso III
16     else x.p = y                                    // caso III
17     RB-Transplant(T, z, y)                          // caso III
18     y.left = z.left                                 // caso III
19     y.left.p = y                                    // caso III
20     y.color = z.color                               // caso III
21 if y-original-color == BLACK
22     RB-Delete-Fixup(T, x)
```

RB-Delete-Fixup(T, x)

```
1  while x ≠ T.root and x.color == BLACK
2      if x == x.p.left
3          w = x.p.right                               // w é o irmão de x
4          if w.color == RED
5              w.color = BLACK                         // caso 1
6              x.p.color = RED                         // caso 1
7              Left-Rotate(T, x.p)                     // caso 1
8              w = x.p.right                           // caso 1
9          if w.left.color == BLACK and w.right.color == BLACK
10             w.color = RED                           // caso 2
11             x = x.p                                 // caso 2
12         else
13             if w.right.color == BLACK
14                 w.left.color = BLACK                // caso 3
15                 w.color = RED                       // caso 3
16                 Right-Rotate(T, w)                  // caso 3
17                 w = x.p.right                       // caso 3
18             w.color = x.p.color                     // caso 4
19             x.p.color = BLACK                       // caso 4
20             w.right.color = BLACK                   // caso 4
21             Left-Rotate(T, x.p)                     // caso 4
22             x = T.root                              // caso 4
23     else  // mesmo que as linhas 3–22, com right e left trocados
44 x.color = BLACK
```

Caso I da remoção (rubro-negra)

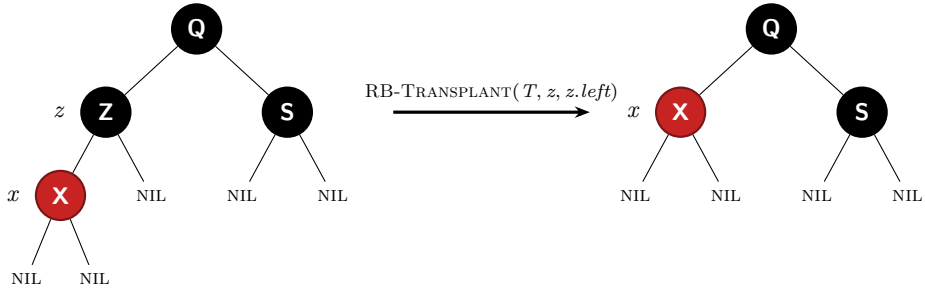
- O filho esquerdo de z é NIL.
- Fazemos $x = z.right$ e substituímos z por x: RB-TRANSPLANT$(T, z, z.right)$.
- Se a cor original de z era preta, chamamos RB-DELETE-FIXUP(T, x).



- Corresponde às linhas 3–5 de RB-DELETE.

Caso II da remoção (rubro-negra)

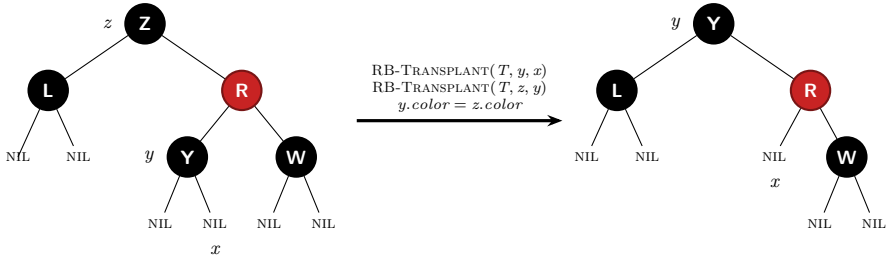
- O filho direito de z é NIL.
- Fazemos $x = z.left$ e substituímos z por x: RB-TRANSPLANT$(T, z, z.left)$.
- Se a cor original de z era preta, chamamos RB-DELETE-FIXUP(T, x).



- Corresponde às linhas 6–8 de RB-DELETE.

Caso III da remoção (rubro-negra)

- Nenhum dos filhos de z é NIL.
- $y = \text{TREE-MINIMUM}(z.right)$ ocupa o lugar de z e herda a cor de z; $x = y.right$ ocupa o lugar de y.
- Se a cor original de y era preta, chamamos $\text{RB-DELETE-FIXUP}(T, x)$.



- Corresponde às linhas 9–22 de RB-DELETE

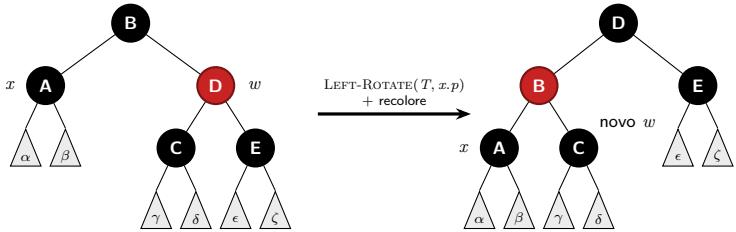
- Sendo w o irmão de x, existem quatro casos:
  - **Caso 1:** w é vermelho;
  - **Caso 2:** w é preto e os dois filhos de w são pretos;
  - **Caso 3:** w é preto, $w.left$ é vermelho e $w.right$ é preto;
  - **Caso 4:** w é preto e $w.right$ é vermelho.

Considerações

1. O laço termina quando x é vermelho ou é a raiz; a linha 44 pinta x de preto.
2. Nas ilustrações a seguir, $\alpha, \beta, \ldots, \zeta$ são subárvores.

Caso 1 da recoloração: RB-Delete-Fixup

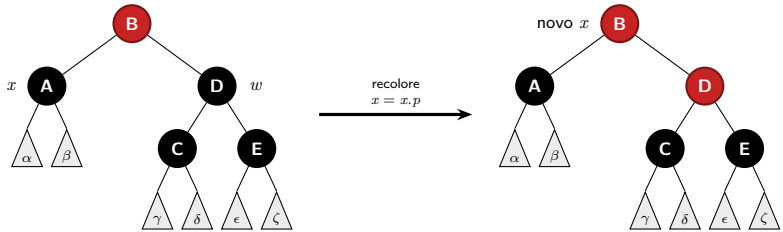
- O irmão w de x é vermelho.
- Trocamos as cores de w e $x.p$ e fazemos Left-Rotate$(T, x.p)$.
- Se o novo irmão de x é preto, então seguimos para o caso 2, 3 ou 4.



- Linhas 4–8. A ilustração considera $x = x.p.left$; o caso $x = x.p.right$ é simétrico.

Caso 2 da recoloração: RB-Delete-Fixup

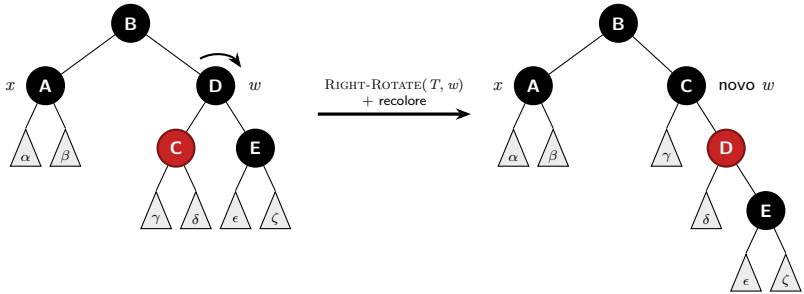
- w é preto e os dois filhos de w são pretos.
- Pintamos w de vermelho e subimos o preto extra: $x = x.p$.
- Se o novo x é vermelho, o laço termina e a linha 44 o pinta de preto.



- Linhas 9–11. $x.p$ pode ter qualquer cor. A ilustração considera $x = x.p.left$; o caso $x = x.p.right$ é simétrico.

Caso 3 da recoloração: RB-Delete-Fixup

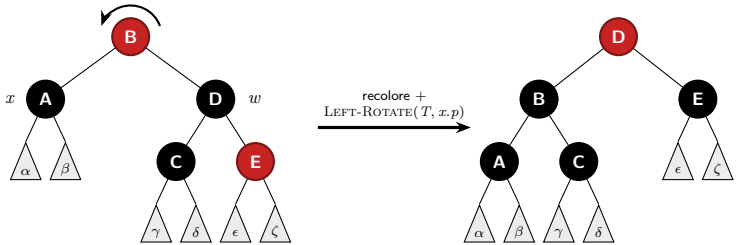
- w é preto, $w.left$ é vermelho e $w.right$ é preto.
- Trocamos as cores de w e $w.left$ e fazemos $\textsc{Right-Rotate}(T, w)$.
- Se o novo irmão de x tem filho direito vermelho, então caso 4.



- Linhas 13–17. $x.p$ pode ter qualquer cor. A ilustração considera $x = x.p.left$; o caso $x = x.p.right$ é simétrico.

Caso 4 da recoloração: RB-Delete-Fixup

- w é preto e $w.right$ é vermelho.
- w recebe a cor de $x.p$; $x.p$ e $w.right$ ficam pretos; Left-Rotate$(T, x.p)$.
- Após isso, $x = T.root$: o laço termina.



- Linhas 18–22. $x.p$ e $w.left$ podem ter qualquer cor. A ilustração considera $x = x.p.left$; o caso $x = x.p.right$ é simétrico.

Roteiro

1. Introdução
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Fechamento

- A escolha de qual estrutura de dados utilizar depende da sua aplicação
- Sua aplicação...
  - Prioriza achar mínimo e máximo? *arrays* ordenados
  - Realiza modificações com muita frequência? *Red-black trees*
  - Prioriza eficiência no acesso? *hash tables*

Estrutura	SEARCH	INSERT	DELETE	MIN/MAX	SUCC/PRED
Array não ordenado	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Array ordenado	$O(\lg n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Stack / queue	—	$O(1)$	$O(1)$	—	—
Lista duplamente ligada	$\Theta(n)$	$O(1)$	$O(1)$	$\Theta(n)$	$\Theta(n)$
Hash table (encadeamento)	$\Theta(1+\alpha)$ médio	$O(1)$	$O(1)$	—	—
Hash table (end. aberto)	$\leq \frac{1}{1-\alpha}$ sondas	$\leq \frac{1}{1-\alpha}$ sondas	—	—	—
Binary search tree	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(h)$
Red-black tree	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$

Obrigado!